

VL-09: LOOP und WHILE Programme I

(Berechenbarkeit und Komplexität, WS 2024)

Peter Rossmanith

WS 2024, RWTH

Wiederholung

Wdh.: Der Satz von Richardson

Eine Funktion heisst **elementar**, wenn sie durch Kombination von Addition, Subtraktion, Multiplikation, Division, Potenzieren, Wurzel ziehen, Logarithmieren, Betragsfunktion und den trigonometrischen Funktionen konstruiert werden kann.

Im Jahr 1968 bewies der Britische Mathematiker Daniel Richardson das folgende Unentscheidbarkeitsresultat:

Satz von Richardson (1968)

Es ist unentscheidbar, ob eine gegebene **elementare** Funktion eine **elementare** Stammfunktion besitzt.

Der Beweis reduziert (im Wesentlichen) das Halteproblem für Turing Maschinen auf das Integrationsproblem.

Im Originalwortlaut (1900):

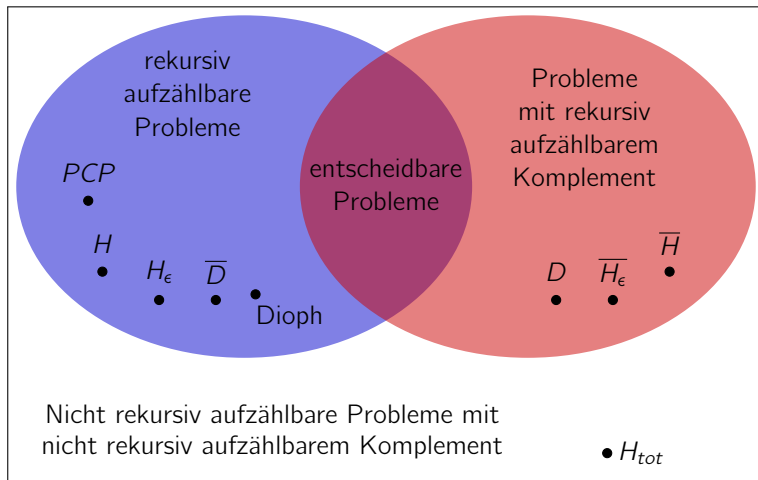
Eine *Diophantische* Gleichung mit irgend welchen Unbekannten und mit ganzen rationalen Zahlencoeffizienten sei vorgelegt: man soll ein Verfahren angeben, nach welchem sich mittelst einer endlichen Anzahl von Operationen entscheiden lässt, ob die Gleichung in ganzen rationalen Zahlen lösbar ist.

$\text{Dioph} = \{ \langle p \rangle \mid p \text{ ist ein Polynom mit ganzzahligen Koeffizienten und mit (mindestens) einer ganzzahligen Nullstelle} \}$

Satz von Matiyasevich (1970)

Es ist unentscheidbar, ob ein (multivariates) ganzzahliges Polynom eine ganzzahlige Nullstelle besitzt.

Wdh.: Berechenbarkeitslandschaft



Definition

Ein Rechnermodell wird als **Turing-mächtig** bezeichnet, wenn jede Funktion, die durch eine TM berechnet werden kann, auch durch dieses Rechnermodell berechnet werden kann.

- Da die Registermaschine (RAM) die Turingmaschine simulieren kann, ist sie Turing-mächtig
- Auch die Mini-RAM (eine schwächere Variante der RAM mit stark eingeschränktem Befehlssatz) ist Turing-mächtig

- Reines HTML (ohne JavaScript; ohne Browser) ist **nicht** Turing-mächtig
- Tabellenkalkulationen (ohne Schleifen) sind **nicht** Turing-mächtig
- Der Lambda Calculus von Alonzo Church ist äquivalent zur TM, und daher Turing-mächtig
- Die μ -rekursiven Funktionen von Stephen Kleene sind äquivalent zur TM, und daher Turing-mächtig
- Alle gängigen höheren Programmiersprachen sind Turing-mächtig: Algol, Pascal, C, FORTRAN, COBOL, Java, Smalltalk, Ada, C++, Python, LISP, Haskell, PROLOG, etc.
- PostScript, Tex, Latex sind Turing-mächtig
- Sogar PowerPoint ist Turing-mächtig (Animated Features)

Vorlesung VL-09

LOOP und WHILE Programme I

- Die Programmiersprache LOOP
- Die Programmiersprache WHILE
- WHILE versus LOOP
- WHILE ist Turing-mächtig
- Die Ackermann Funktion

Die Programmiersprache LOOP

Programmiersprache LOOP

Wir betrachten eine einfache Programmiersprache namens LOOP, deren Programme aus den folgenden syntaktischen Komponenten aufgebaut sind:

- Variablen: x_0 x_1 x_2 x_3 ...
- Konstanten: 0 und 1
- Symbole: $:=$ + ;
- Schlüsselwörter: LOOP DO ENDLOOP

Die Syntax von LOOP ist induktiv definiert.

Induktive Definition / Induktionsanfang:

Zuweisungen

Für alle Variablen x_i und x_j und für jede Konstante $c \in \{0, 1\}$ ist die Zuweisung

$$x_i := x_j + c$$

ein LOOP Programm.

Anmerkung: Auch das leere Programm ist ein LOOP Programm.

Induktive Definition / Induktionsschritt:

Hintereinanderausführung

Falls P_1 und P_2 LOOP Programme sind, so ist auch

$$P_1; P_2$$

ein LOOP Programm.

Induktive Definition / Induktionsschritt:

LOOP-Konstrukt

Falls P ein LOOP Programm ist, so ist auch

`LOOP  $x_i$  DO  $P$  ENDLOOP`

ein LOOP Programm.

LOOP / Semantik (1)

- Die Eingabe ist in den Variablen $x_1, \dots, x_m$ enthalten.
Alle anderen Variablen werden mit 0 initialisiert.
- Das Resultat eines LOOP Programms ist die Zahl, die sich am Ende der Abbarbeitung in der Variablen x_0 ergibt.

LOOP / Semantik (1)

- Die Eingabe ist in den Variablen $x_1, \dots, x_m$ enthalten.
Alle anderen Variablen werden mit 0 initialisiert.
- Das Resultat eines LOOP Programms ist die Zahl, die sich am Ende der Abbarbeitung in der Variablen x_0 ergibt.

LOOP Programme der Form $x_j := x_j + c$
sind Zuweisungen des Wertes $x_j + c$ an die Variable x_j .

LOOP / Semantik (1)

- Die Eingabe ist in den Variablen $x_1, \dots, x_m$ enthalten.
Alle anderen Variablen werden mit 0 initialisiert.
- Das Resultat eines LOOP Programms ist die Zahl, die sich am Ende der Abbarbeitung in der Variablen x_0 ergibt.

LOOP Programme der Form $x_j := x_j + c$
sind Zuweisungen des Wertes $x_j + c$ an die Variable x_j .

In einem LOOP Programm $P_1; P_2$
wird zunächst P_1 und danach P_2 ausgeführt.

LOOP / Semantik (1)

- Die Eingabe ist in den Variablen $x_1, \dots, x_m$ enthalten.
Alle anderen Variablen werden mit 0 initialisiert.
- Das Resultat eines LOOP Programms ist die Zahl, die sich am Ende der Abbarbeitung in der Variablen x_0 ergibt.

LOOP Programme der Form $x_j := x_j + c$
sind Zuweisungen des Wertes $x_j + c$ an die Variable x_j .

In einem LOOP Programm $P_1; P_2$
wird zunächst P_1 und danach P_2 ausgeführt.

Das Programm **LOOP** x_i **DO** P **ENDLOOP** hat folgende Bedeutung:
 P wird x_i -mal hintereinander ausgeführt.

(Nur der Wert von x_i zu Beginn der Schleife ist relevant. Ändert sich der Wert von x_i im Inneren von P , so hat dies keinen Einfluss auf die Anzahl der Wiederholungen.)

LOOP / Semantik (2)

Ein LOOP Programm P mit k Variablen
berechnet eine k -stellige Funktion der Form $[P]: \mathbb{N}^k \rightarrow \mathbb{N}^k$.

Ist P die Zuweisung $x_i := x_j + c$,
so ist $[P](r_0, \dots, r_{k-1}) = (r_0, \dots, r_{i-1}, r_j + c, r_{i+1}, \dots, r_{k-1})$.

Ist $P = P_1; P_2$ eine Hintereinanderausführung,
so ist $[P](r_0, \dots, r_{k-1}) = [P_2]([P_1](r_0, \dots, r_{k-1}))$.

Ist $P = \text{LOOP } x_i \text{ DO } Q \text{ ENDLOOP}$ ein LOOP-Konstrukt,
so gilt $[P](r_0, \dots, r_{k-1}) = [Q]^{r_i}(r_0, \dots, r_{k-1})$.

LOOP: Beispiele und Macros

LOOP Programme / Beispiele (1)

Das folgende Programm simuliert die Zuweisung $x_j := x_i$.

Beispiel A

$$x_j := x_i + 0$$

Es sei x_{zero} eine Dummy-Variable, die mit 0 initialisiert wird und deren Wert nie verändert wird. Das folgende $(c + 1)$ -zeilige Programm simuliert die Zuweisung $x_j := c$ eines konstanten Wertes $c \geq 0$ an eine Variable.

Beispiel B

```

$$\begin{aligned} x_j &:= x_{\text{zero}}; \\ x_j &:= x_j + 1; \\ x_j &:= x_j + 1; \\ &\vdots \\ x_j &:= x_j + 1; \end{aligned}$$

```

Beispiel C

```
 $x_0 := x_1;$   
LOOP  $x_2$  DO  $x_0 := x_0 + 1$  ENDLOOP
```

Beispiel C

```
 $x_0 := x_1;$   
LOOP  $x_2$  DO  $x_0 := x_0 + 1$  ENDLOOP
```

Dieses Programm berechnet die Addition $x_0 := x_1 + x_2$

Beispiel C

```
 $x_0 := x_1;$   
LOOP  $x_2$  DO  $x_0 := x_0 + 1$  ENDLOOP
```

Dieses Programm berechnet die Addition $x_0 := x_1 + x_2$

Beispiel D

```
 $x_0 := 0;$   
LOOP  $x_2$  DO  $x_0 := x_0 + x_1$  ENDLOOP
```

Beispiel C

```
 $x_0 := x_1;$   
LOOP  $x_2$  DO  $x_0 := x_0 + 1$  ENDLOOP
```

Dieses Programm berechnet die Addition $x_0 := x_1 + x_2$

Beispiel D

```
 $x_0 := 0;$   
LOOP  $x_2$  DO  $x_0 := x_0 + x_1$  ENDLOOP
```

Dieses Programm berechnet die Multiplikation $x_0 := x_1 \cdot x_2$

Übung

Skizzieren Sie LOOP Programme, die die folgenden Operationen berechnen:

- Die (modifizierte) Subtraktion $x_0 := x_1 \dot{-} x_2$.
Für $x_1 < x_2$ erhält x_0 den Wert 0; andernfalls den Wert $x_1 - x_2$
- Die Division ohne Rest $x_0 := x_1 \text{ DIV } x_2$
- Die Modulo-Operation $x_0 := x_1 \text{ MOD } x_2$

LOOP Programme / Beispiele (4)

Es seien P_1 und P_2 LOOP Programme, in denen die drei Variablen x_1 , x_2 und x_3 nicht vorkommen.

Beispiel E

```
 $x_2 := 1; \ x_3 := 0;$   
LOOP  $x_1$  DO  $x_2 := 0; \ x_3 := 1$  ENDLOOP;  
LOOP  $x_2$  DO  $P_1$  ENDLOOP;  
LOOP  $x_3$  DO  $P_2$  ENDLOOP
```

LOOP Programme / Beispiele (4)

Es seien P_1 und P_2 LOOP Programme, in denen die drei Variablen x_1 , x_2 und x_3 nicht vorkommen.

Beispiel E

```
x2 := 1; x3 := 0;  
LOOP x1 DO x2 := 0; x3 := 1 ENDLOOP;  
LOOP x2 DO P1 ENDLOOP;  
LOOP x3 DO P2 ENDLOOP
```

Dieses Programm entspricht dem Konstrukt:

IF $x_1 = 0$ **THEN** P_1 **ELSE** P_2 **ENDIF**

LOOP Programme / Beispiele (4)

Es seien P_1 und P_2 LOOP Programme, in denen die drei Variablen x_1 , x_2 und x_3 nicht vorkommen.

Beispiel E

```
x2 := 1; x3 := 0;  
LOOP x1 DO x2 := 0; x3 := 1 ENDLOOP;  
LOOP x2 DO P1 ENDLOOP;  
LOOP x3 DO P2 ENDLOOP
```

Dieses Programm entspricht dem Konstrukt:

IF $x_1 = 0$ **THEN** P_1 **ELSE** P_2 **ENDIF**

Übung

Skizzieren Sie ein LOOP Programm,
das **“IF** $x_1 = c$ **THEN** P_1 **ELSE** P_2 **ENDIF”** simuliert.

Die Programmiersprache WHILE

Programmiersprache WHILE

Die Programme der Programmiersprache WHILE sind aus den folgenden syntaktischen Komponenten aufgebaut:

- Variablen: x_0 x_1 x_2 x_3 ...
- Konstanten: 0 und 1
- Symbole: $:=$ + ; $\neq$
- Schlüsselwörter: LOOP DO ENDLOOP **WHILE ENDWHILE**

- Die Syntax von WHILE ist induktiv definiert, und stimmt weitgehend mit der Syntax von LOOP überein.
- Zuweisungen $x_i := x_j + c$ und die Hintereinanderausführung $P_1; P_2$ sind genau wie in LOOP definiert.
- Der Hauptunterschied zu LOOP besteht im Schleifen-Konstrukt.

WHILE-Konstrukt

Falls P ein WHILE Programm ist und x_i eine Variable, so ist auch

$\text{WHILE } x_i \neq 0 \text{ DO } P \text{ ENDWHILE}$

ein WHILE Programm.

Analog zu LOOP Programmen:

- Die Eingabe ist in den Variablen $x_1, \dots, x_m$ enthalten.
Alle anderen Variablen werden mit 0 initialisiert.
- Das Resultat eines WHILE Programms ist die Zahl, die sich am Ende der Abbarbeitung in der Variablen x_0 ergibt.

WHILE / Semantik (2)

Das Programm `WHILE  $x_i \neq 0$  DO  $P$  ENDWHILE` hat folgende Bedeutung:

P wird solange ausgeführt, bis x_i den Wert 0 erreicht.

Ein WHILE Programm P mit k Variablen

berechnet eine k -stellige Funktion der Form $[P]: \mathbb{N}^k \rightarrow \mathbb{N}^k$.

Ist $P = \text{WHILE } x_i \neq 0 \text{ DO } Q \text{ ENDWHILE}$ ein WHILE-Konstrukt, so ist $[P](r_0, \dots, r_{k-1}) = [Q]^\ell(r_0, \dots, r_{k-1})$ für die kleinste Zahl ℓ , für die die i -te Komponente von $[Q]^\ell(r_0, \dots, r_{k-1})$ gleich 0 ist.

Falls solch ein ℓ nicht existiert, so ist $[P](r_0, \dots, r_{k-1})$ undefiniert.

WHILE versus LOOP

WHILE versus LOOP (1)

Jede LOOP-berechenbare Funktion $f: \mathbb{N}^k \rightarrow \mathbb{N}$ ist auch WHILE-berechenbar.

Bemerkung

Die LOOP-Schleife

`LOOP  $x_i$  DO  $P$  ENDLOOP`

kann auch durch die folgende WHILE-Schleife simuliert werden:

`$y := x_i$`

`WHILE  $y \neq 0$  DO  $y := y - 1$ ;  $P$  ENDWHILE`

WHILE versus LOOP (2a)

Es gibt WHILE Programme, die nicht terminieren:

Beispiel

```
 $x_1 := 1;$   
WHILE  $x_1 \neq 0$  DO  $x_1 := x_1 + 1$  ENDWHILE
```

WHILE versus LOOP (2b)

LOOP Programme terminieren immer:

Satz

Jedes LOOP Programm hält auf jeder möglichen Eingabe nach endlich vielen Schritten an.

Beweis: Durch Induktion über den Aufbau des Programms.

- Zuweisungen
- Hintereinanderausführung $P = P_1; P_2$
- LOOP-Konstrukt $P = \text{LOOP } x_i \text{ DO } Q \text{ ENDLOOP}$

WHILE versus LOOP (3)

Wir werden zeigen:

Satz (wird heute bewiesen)

Die Programmiersprache WHILE ist **Turing-mächtig**.

In anderen Worten: Jede berechenbare Funktion kann von einem WHILE Programm berechnet werden.

Satz (wird in der nächsten Vorlesung bewiesen)

Die Programmiersprache LOOP ist **nicht Turing-mächtig**.

In anderen Worten: Es existiert eine berechenbare **totale** Funktion, die von keinem LOOP Programm berechnet werden kann.

Mächtigkeit von WHILE

Satz

Die Programmiersprache WHILE ist Turing-mächtig.

Beweis:

Wir zeigen, dass jede Funktion, die durch eine TM berechnet werden kann, auch durch ein WHILE Programm berechnet werden kann.

Simulation von TM durch WHILE

Wir betrachten eine TM M .

- Zustandsmenge $Q = \{q_0, \dots, q_t\}$
- Der Anfangszustand ist q_1 , und der Endzustand ist q_0
- TM im Zustand $q_i \iff$ WHILE Variable **Zustand** = i

Simulation von TM durch WHILE

Wir betrachten eine TM M .

- Zustandsmenge $Q = \{q_0, \dots, q_t\}$
- Der Anfangszustand ist q_1 , und der Endzustand ist q_0
- TM im Zustand $q_i \iff$ WHILE Variable **Zustand** = i
- Bandalphabet $\Gamma = \{1, 2, B\}$
- WHILE kodiert
 - Buchstaben 1 durch Dezimalziffer 1,
 - Buchstaben 2 durch Dezimalziffer 2, und
 - Buchstaben B durch Dezimalziffer 0.

Simulation von TM durch WHILE

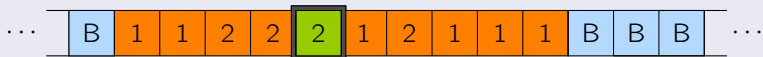
Wir betrachten eine TM M .

- Zustandsmenge $Q = \{q_0, \dots, q_t\}$
- Der Anfangszustand ist q_1 , und der Endzustand ist q_0
- TM im Zustand $q_i \iff$ WHILE Variable **Zustand** = i

- Bandalphabet $\Gamma = \{1, 2, B\}$
- WHILE kodiert
 - Buchstaben 1 durch Dezimalziffer 1 ,
 - Buchstaben 2 durch Dezimalziffer 2 , und
 - Buchstaben B durch Dezimalziffer 0 .

- Die Werte in allen WHILE Variablen werden im Folgenden als Dezimalzahlen interpretiert

Simulierte Turingmaschine M :

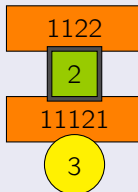
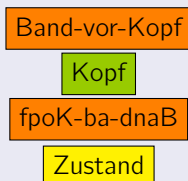


δ	1	2	B
q_1			
q_2			
q_3		$(q_2, 1, R)$	



Entsprechende Konfiguration: $1122 q_3 212111$

Die vier entsprechenden Variablen im WHILE Programm:



Variable **BandLinks**
Variable **UntermKopf**
Variable **BandRechts**
Variable **Zustand**

Simulation von TM durch WHILE

Jeder Rechenschritt von M wird durch einige WHILE-Befehle simuliert.

Jeder Rechenschritt der TM besteht (gemäss Überföhrungsfunktion) aus

- (A) Update von Zustand
- (B) Update von Symbol unterm Kopf
- (C) Bewegung des Kopfes L,R,N

Beginn der Rechenschritt Simulation

- Aktueller Zustand steht in der Variablen `Zustand`
- Aktuelles Symbol unterm Kopf steht in der Variablen `UntermKopf`

Simulation von TM durch WHILE (A)

Jeder Rechenschritt der TM besteht (gemäss Überföhrungsfunktion) aus

(A) Update von Zustand

(B) Update von Symbol unterm Kopf

(C) Bewegung des Kopfes L,R,N

Der Zustand wird auf den neuen Zustand q_i gesetzt,
indem man das folgende Programmstück ausföhrt:

Zustand := i ;

Simulation von TM durch WHILE (B)

Jeder Rechenschritt der TM besteht (gemäss Überföhrungsfunktion) aus

- (A) Update von Zustand
- (B) Update von Symbol unterm Kopf**
- (C) Bewegung des Kopfes L,R,N

Das Symbol unterm Kopf wird auf neues Symbol $\sigma \in \{0, 1, 2\}$ gesetzt, indem man das folgende Programmstück ausföhrt:

UntermKopf := σ ;

Simulation von TM durch WHILE (C/links)

Jeder Rechenschritt der TM besteht (gemäss Überföhrungsfunktion) aus

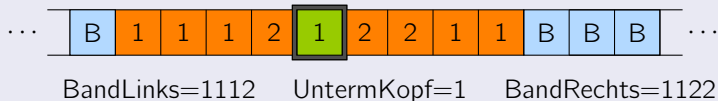
- (A) Update von Zustand
- (B) Update von Symbol unterm Kopf
- (C) **Bewegung des Kopfes L,R,N**

Der Kopf wird einen Schritt nach links (L) bewegt,
indem man das folgende Programmstück ausföhrt:

```
BandRechts := 10 × BandRechts + UntermKopf;  
UntermKopf := BandLinks MOD 10;  
BandLinks := BandLinks DIV 10;
```


Illustration: Kopfbewegung nach links

Alte Situation vor Bewegung



$$\text{BandRechts} := 10 \times \text{BandRechts} + \text{UntermKopf} = 11220 + 1$$

$$\text{UntermKopf} := \text{BandLinks} \text{ MOD } 10 = 2$$

$$\text{BandLinks} := \text{BandLinks} \text{ DIV } 10 = 111$$

Neue Situation nach Bewegung



Simulation von TM durch WHILE (C/rechts)

Jeder Rechenschritt der TM besteht (gemäss Überföhrungsfunktion) aus

- (A) Update von Zustand
- (B) Update von Symbol unterm Kopf
- (C) **Bewegung des Kopfes L,R,N**

Der Kopf wird einen Schritt nach rechts (R) bewegt,
indem man das folgende Programmstück ausföhrt:

```
BandLinks := 10×BandLinks + UntermKopf;  
UntermKopf := BandRechts MOD 10;  
BandRechts := BandRechts DIV 10;
```

Simulation von TM durch WHILE (C/nichts)

Jeder Rechenschritt der TM besteht (gemäss Überföhrungsfunktion) aus

- (A) Update von Zustand
- (B) Update von Symbol unterm Kopf
- (C) **Bewegung des Kopfes L,R,N**

Der Kopf wird nicht bewegt (N),
indem man gar nichts macht und alle Variablen unverändert lässt:



Grobstruktur der Simulation (1)

Schlussendlich die Grobstruktur der Simulation:

- Initialisierung
- Schleifenstruktur

Initialisierung

Zustand := 1;

BandLinks := 0;

UntermKopf := *Erstes Symbol im Eingabewort (als Dezimalziffer)*;

BandRechts := *Restliches Eingabewort (dezimal & gespiegelt)*;

Grobstruktur der Simulation (2)

Die äussere Schleife

```
WHILE Zustand  $\neq$  0 DO
    IF Zustand=1 AND UntermKopf=0 THEN Schritt ENDIF;
    IF Zustand=1 AND UntermKopf=1 THEN Schritt ENDIF;
    IF Zustand=1 AND UntermKopf=2 THEN Schritt ENDIF;

    IF Zustand=2 AND UntermKopf=0 THEN Schritt ENDIF;
    IF Zustand=2 AND UntermKopf=1 THEN Schritt ENDIF;
    IF Zustand=2 AND UntermKopf=2 THEN Schritt ENDIF;

    IF Zustand=3 AND UntermKopf=0 THEN Schritt ENDIF;
    :           :           :           :
    :           :           :           :
    IF Zustand= $t$  AND UntermKopf=0 THEN Schritt ENDIF;
    IF Zustand= $t$  AND UntermKopf=1 THEN Schritt ENDIF;
    IF Zustand= $t$  AND UntermKopf=2 THEN Schritt ENDIF;
ENDWHILE
```

Die Ackermann Funktion

Ackermann Funktion: Definition

Definition

Die Ackermann Funktion $A : \mathbb{N}^2 \rightarrow \mathbb{N}$ ist folgendermassen definiert:

$$\begin{array}{lll} A(0, n) & = & n + 1 & \text{für } n \geq 0 \\ A(m + 1, 0) & = & A(m, 1) & \text{für } m \geq 0 \\ A(m + 1, n + 1) & = & A(m, A(m + 1, n)) & \text{für } m, n \geq 0 \end{array}$$

Ackermann Funktion: Beispiele (1)

$$\begin{array}{lll} A(0, n) & = & n + 1 & \text{für } n \geq 0 \\ A(m + 1, 0) & = & A(m, 1) & \text{für } m \geq 0 \\ A(m + 1, n + 1) & = & A(m, A(m + 1, n)) & \text{für } m, n \geq 0 \end{array}$$

Ein paar Beispiele für $m = 1$:

$$A(1, 0) = A(0, 1) = 2$$

Ackermann Funktion: Beispiele (1)

$$\begin{array}{lll} A(0, n) & = & n + 1 & \text{für } n \geq 0 \\ A(m + 1, 0) & = & A(m, 1) & \text{für } m \geq 0 \\ A(m + 1, n + 1) & = & A(m, A(m + 1, n)) & \text{für } m, n \geq 0 \end{array}$$

Ein paar Beispiele für $m = 1$:

$$A(1, 0) = A(0, 1) = 2$$

$$A(1, 1) = A(0, A(1, 0)) = A(1, 0) + 1 = 3$$

$$A(1, 2) = A(0, A(1, 1)) = A(1, 1) + 1 = 4$$

$$A(1, 3) = A(0, A(1, 2)) = A(1, 2) + 1 = 5$$

Ackermann Funktion: Beispiele (1)

$$\begin{array}{lll} A(0, n) & = & n + 1 & \text{für } n \geq 0 \\ A(m + 1, 0) & = & A(m, 1) & \text{für } m \geq 0 \\ A(m + 1, n + 1) & = & A(m, A(m + 1, n)) & \text{für } m, n \geq 0 \end{array}$$

Ein paar Beispiele für $m = 1$:

$$A(1, 0) = A(0, 1) = 2$$

$$A(1, 1) = A(0, A(1, 0)) = A(1, 0) + 1 = 3$$

$$A(1, 2) = A(0, A(1, 1)) = A(1, 1) + 1 = 4$$

$$A(1, 3) = A(0, A(1, 2)) = A(1, 2) + 1 = 5$$

Beobachtung

$$A(1, n) =$$

Ackermann Funktion: Beispiele (1)

$$\begin{array}{lll} A(0, n) & = & n + 1 & \text{für } n \geq 0 \\ A(m + 1, 0) & = & A(m, 1) & \text{für } m \geq 0 \\ A(m + 1, n + 1) & = & A(m, A(m + 1, n)) & \text{für } m, n \geq 0 \end{array}$$

Ein paar Beispiele für $m = 1$:

$$A(1, 0) = A(0, 1) = 2$$

$$A(1, 1) = A(0, A(1, 0)) = A(1, 0) + 1 = 3$$

$$A(1, 2) = A(0, A(1, 1)) = A(1, 1) + 1 = 4$$

$$A(1, 3) = A(0, A(1, 2)) = A(1, 2) + 1 = 5$$

Beobachtung

$$A(1, n) = n + 2$$

Ackermann Funktion: Beispiele (2)

$$\begin{array}{lll} A(0, n) & = & n + 1 & \text{für } n \geq 0 \\ A(m + 1, 0) & = & A(m, 1) & \text{für } m \geq 0 \\ A(m + 1, n + 1) & = & A(m, A(m + 1, n)) & \text{für } m, n \geq 0 \end{array}$$

Ein paar Beispiele für $m = 2$:

$$A(2, 0) = A(1, 1) = 3$$

Ackermann Funktion: Beispiele (2)

$$\begin{array}{lll} A(0, n) & = & n + 1 & \text{für } n \geq 0 \\ A(m + 1, 0) & = & A(m, 1) & \text{für } m \geq 0 \\ A(m + 1, n + 1) & = & A(m, A(m + 1, n)) & \text{für } m, n \geq 0 \end{array}$$

Ein paar Beispiele für $m = 2$:

$$A(2, 0) = A(1, 1) = 3$$

$$A(2, 1) = A(1, A(2, 0)) = A(2, 0) + 2 = 5$$

$$A(2, 2) = A(1, A(2, 1)) = A(2, 1) + 2 = 7$$

$$A(2, 3) = A(1, A(2, 2)) = A(2, 2) + 2 = 9$$

Beobachtung

$$A(2, n) =$$

Ackermann Funktion: Beispiele (2)

$$\begin{array}{lll} A(0, n) & = & n + 1 & \text{für } n \geq 0 \\ A(m + 1, 0) & = & A(m, 1) & \text{für } m \geq 0 \\ A(m + 1, n + 1) & = & A(m, A(m + 1, n)) & \text{für } m, n \geq 0 \end{array}$$

Ein paar Beispiele für $m = 2$:

$$A(2, 0) = A(1, 1) = 3$$

$$A(2, 1) = A(1, A(2, 0)) = A(2, 0) + 2 = 5$$

$$A(2, 2) = A(1, A(2, 1)) = A(2, 1) + 2 = 7$$

$$A(2, 3) = A(1, A(2, 2)) = A(2, 2) + 2 = 9$$

Beobachtung

$$A(2, n) = 2n + 3$$

Ackermann Funktion: Beispiele (3)

$$\begin{array}{lll} A(0, n) & = & n + 1 & \text{für } n \geq 0 \\ A(m + 1, 0) & = & A(m, 1) & \text{für } m \geq 0 \\ A(m + 1, n + 1) & = & A(m, A(m + 1, n)) & \text{für } m, n \geq 0 \end{array}$$

Und ein paar Beispiele für $m = 3$:

$$A(3, 0) = A(2, 1) = 5$$

Ackermann Funktion: Beispiele (3)

$$\begin{array}{lll} A(0, n) & = & n + 1 & \text{für } n \geq 0 \\ A(m + 1, 0) & = & A(m, 1) & \text{für } m \geq 0 \\ A(m + 1, n + 1) & = & A(m, A(m + 1, n)) & \text{für } m, n \geq 0 \end{array}$$

Und ein paar Beispiele für $m = 3$:

$$A(3, 0) = A(2, 1) = 5$$

$$A(3, 1) = A(2, A(3, 0)) = 2 \cdot A(3, 0) + 3 = 13$$

$$A(3, 2) = A(2, A(3, 1)) = 2 \cdot A(3, 1) + 3 = 29$$

$$A(3, 3) = A(2, A(3, 2)) = 2 \cdot A(3, 2) + 3 = 61$$

Beobachtung

$$A(3, n) =$$

Ackermann Funktion: Beispiele (3)

$$\begin{array}{lll} A(0, n) & = & n + 1 & \text{für } n \geq 0 \\ A(m + 1, 0) & = & A(m, 1) & \text{für } m \geq 0 \\ A(m + 1, n + 1) & = & A(m, A(m + 1, n)) & \text{für } m, n \geq 0 \end{array}$$

Und ein paar Beispiele für $m = 3$:

$$A(3, 0) = A(2, 1) = 5$$

$$A(3, 1) = A(2, A(3, 0)) = 2 \cdot A(3, 0) + 3 = 13$$

$$A(3, 2) = A(2, A(3, 1)) = 2 \cdot A(3, 1) + 3 = 29$$

$$A(3, 3) = A(2, A(3, 2)) = 2 \cdot A(3, 2) + 3 = 61$$

Beobachtung

$$A(3, n) = 2^{n+3} - 3$$

Ackermann Funktion: Zusammenfassung der Beispiele

Wenn man den ersten Parameter fixiert:

- $A(1, n) = n + 2$
- $A(2, n) = 2n + 3$
- $A(3, n) = 2^{n+3} - 3$
- $A(4, n) = \underbrace{2^{2^{\cdot^{\cdot^2}}}}_{\substack{n+3 \text{ viele} \\ \text{Zweien}}} - 3$

Bereits der Wert $A(4, 2) = 2^{65536} - 3$
ist grösser als die Anzahl aller Atome im Weltraum.

“Wovon man nicht sprechen kann, darüber muss man schweigen.”

Exkurs: UpArrow-Notation (1)

- Addition ist iterierte Nachfolgerbildung:

$$\underbrace{S(S(\dots(S(a)\dots))}_{b \text{ mal}} = a + b$$

Exkurs: UpArrow-Notation (1)

- Addition ist iterierte Nachfolgerbildung:

$$\underbrace{S(S(\dots(S(a)\dots))}_{b \text{ mal}} = a + b$$

- Multiplikation ist iterierte Addition:

$$\underbrace{a + \dots + a}_{b \text{ mal}} = a \times b$$

Exkurs: UpArrow-Notation (1)

- Addition ist iterierte Nachfolgerbildung:

$$\underbrace{S(S(\dots(S(a)\dots))}_{b \text{ mal}} = a + b$$

- Multiplikation ist iterierte Addition:

$$\underbrace{a + \dots + a}_{b \text{ mal}} = a \times b$$

- Potenzierung ist iterierte Multiplikation:

$$\underbrace{a \times \dots \times a}_{b \text{ mal}} = a^b =: a \uparrow b$$

Exkurs: UpArrow-Notation (1)

- Addition ist iterierte Nachfolgerbildung:

$$\underbrace{S(S(\dots(S(a)\dots))}_{b \text{ mal}} = a + b$$

- Multiplikation ist iterierte Addition:

$$\underbrace{a + \dots + a}_{b \text{ mal}} = a \times b$$

- Potenzierung ist iterierte Multiplikation:

$$\underbrace{a \times \dots \times a}_{b \text{ mal}} = a^b =: a \uparrow b$$

- Der Potenzturm ist iterierte Potenzierung:

$$\underbrace{a^{a^{\dots^a}}}_{b \text{ mal}} =: a \uparrow\uparrow b$$

Exkurs: UpArrow-Notation (1)

- Addition ist iterierte Nachfolgerbildung:

$$\underbrace{S(S(\dots(S(a)\dots))}_{b \text{ mal}} = a + b$$

- Multiplikation ist iterierte Addition:

$$\underbrace{a + \dots + a}_{b \text{ mal}} = a \times b$$

- Potenzierung ist iterierte Multiplikation:

$$\underbrace{a \times \dots \times a}_{b \text{ mal}} = a^b =: a \uparrow b$$

- Der Potenzturm ist iterierte Potenzierung:

$$\underbrace{a^{a^{\dots^a}}}_{b \text{ mal}} =: a \uparrow\uparrow b$$

- Wiederholte Potenzturmbildung

gibt $\underbrace{a \uparrow\uparrow a \uparrow\uparrow \dots \uparrow\uparrow a}_{b \text{ mal}} =: a \uparrow\uparrow\uparrow b$

Exkurs: UpArrow-Notation (2)

Definition (UpArrow-Notation von Donald Knuth)

$$a \uparrow^m b := \begin{cases} 1 & \text{wenn } b = 0 \\ a \cdot b & \text{wenn } m = 0 \\ a^b & \text{wenn } m = 1 \\ a \uparrow^{m-1} (a \uparrow^m (b - 1)) & \text{sonst} \end{cases}$$

Exkurs: UpArrow-Notation (2)

Definition (UpArrow-Notation von Donald Knuth)

$$a \uparrow^m b := \begin{cases} 1 & \text{wenn } b = 0 \\ a \cdot b & \text{wenn } m = 0 \\ a^b & \text{wenn } m = 1 \\ a \uparrow^{m-1} (a \uparrow^m (b - 1)) & \text{sonst} \end{cases}$$

Es gilt:

- $A(1, n) = 2 + (n + 3) - 3$
- $A(2, n) = 2 \cdot (n + 3) - 3$
- $A(3, n) = 2 \uparrow (n + 3) - 3$
- $A(4, n) = 2 \uparrow\uparrow (n + 3) - 3$
- $A(5, n) = 2 \uparrow\uparrow\uparrow (n + 3) - 3$
- $\vdots$
- $A(m, n) = 2 \uparrow^{m-2} (n + 3) - 3 \quad \text{für } m \geq 2$